

Автоматизация процесса контроля знаний учащихся

1. Введение

В современном образовательном процессе автоматизация контроля знаний становится неотъемлемой частью эффективной организации обучения. Традиционные методы проверки знаний требуют значительных временных затрат преподавателей, подвержены субъективным факторам и затрудняют систематический анализ успеваемости учащихся.

Актуальность разработки системы автоматизированного контроля знаний обусловлена следующими факторами:

- Необходимость объективной оценки знаний учащихся
- Потребность в быстром получении статистики успеваемости
- Экономия времени преподавателей на проверку тестов
- Возможность дистанционного проведения тестирования
- Хранение и анализ исторических данных об успеваемости

В рамках данного проекта разработано полноценное приложение для автоматизированного контроля знаний учащихся с использованием языка программирования Python, системы управления базами данных SQLite и библиотеки Tkinter для создания графического интерфейса.

2. Постановка задачи

Цель разработки: Создание информационной системы для автоматизации процесса контроля знаний учащихся, включающей следующие функциональные возможности:

- Регистрация предметов и тестов
- Добавление и учёт учащихся
- Проведение тестирования
- Анализ результатов

Требования к системе:

1. Технические требования:

- Хранение данных в реляционной базе данных SQLite
- Графический интерфейс пользователя на базе библиотеки Tkinter
- Язык программирования Python 3.7+
- Кроссплатформенность (Windows, Linux, macOS)

2. Функциональные требования:

- Управление учебными предметами (добавление, просмотр, удаление)
- Управление вопросами тестов (создание вопросов с 4 вариантами ответов)

- Учёт учащихся (регистрация, хранение информации)
- Проведение тестирования в интерактивном режиме
- Автоматический подсчёт результатов
- Поиск и анализ результатов по учащемуся и дате

Структура данных системы:

1. Предмет (Subject):

- Уникальный идентификатор (subject_id)
- Название предмета (subject_name)
- Имя таблицы вопросов (table_name)

2. Вопрос (Question):

- Уникальный идентификатор (question_id)
- Номер вопроса (question_number)
- Текст вопроса (question_text)
- Вариант ответа A (variant_a)
- Вариант ответа B (variant_b)
- Вариант ответа C (variant_c)
- Вариант ответа D (variant_d)
- Правильный ответ (correct_answer)

3. Учащийся (Student):

- Уникальный идентификатор (student_id)
- Фамилия (last_name)
- Имя (first_name)
- Отчество (patronymic)
- Группа (group_name)
- Дата рождения (birth_date)

4. Результат (Result):

- Уникальный идентификатор (result_id)
- ID учащегося (student_id)
- ID предмета (subject_id)
- Количество баллов (score)
- Общее количество вопросов (total_questions)
- Процент правильных ответов (percentage)
- Дата тестирования (test_date)

Основные правила бизнес-логики:

- Один тест содержит множество вопросов
- Тесты привязаны к конкретным предметам
- Каждый студент может пройти тест по предмету только один раз
- Поиск результатов осуществляется по учащемуся и дате проведения

3. Проектирование системы

i. Архитектура системы

Система построена по клиент-серверной архитектуре с локальным хранением данных:

Backend (Серверная часть):

- Язык программирования: Python 3.7+
- База данных: SQLite 3
- Модуль работы с данными: knowledgecontrol.py
- Класс: KnowledgeControlSystem

Frontend (Клиентская часть):

- Библиотека GUI: Tkinter
- Модуль интерфейса: app_ui.py
- Класс: KnowledgeControlApp

Архитектурные преимущества:

- Полная автономность работы (не требует сетевого подключения)
- Простота развёртывания (только интерпретатор Python)
- Надёжное хранение данных в SQLite
- Интуитивно понятный графический интерфейс

ii. База данных

База данных SQLite состоит из следующих таблиц:

Таблица students (учащиеся):

```
CREATE TABLE students (  
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    patronymic TEXT,  
    group_name TEXT NOT NULL,  
    birth_date TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Таблица subjects (предметы):

```
CREATE TABLE subjects (  
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    subject_name TEXT UNIQUE NOT NULL,  
    table_name TEXT UNIQUE NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Таблица results (результаты тестирования):

```
CREATE TABLE results (  
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_id INTEGER NOT NULL,  
    subject_id INTEGER NOT NULL,  
    score INTEGER NOT NULL,  
    total_questions INTEGER NOT NULL,  
    percentage REAL NOT NULL,
```

```
test_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (student_id) REFERENCES students(student_id),  
FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)  
);
```

Таблицы вопросов (создаются динамически для каждого предмета):

```
CREATE TABLE questions_<subjectname> (  
question_id INTEGER PRIMARY KEY AUTOINCREMENT,  
question_number INTEGER NOT NULL,  
question_text TEXT NOT NULL,  
variant_a TEXT NOT NULL,  
variant_b TEXT NOT NULL,  
variant_c TEXT NOT NULL,  
variant_d TEXT NOT NULL,  
correct_answer TEXT NOT NULL CHECK(correct_answer IN ('A', 'B', 'C', 'D')),  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Особенности БД:

- Динамическое создание таблиц вопросов для каждого предмета
- Использование внешних ключей для обеспечения целостности данных
- Автоматическое ведение временных меток
- Ограничения для предотвращения дублирования

iii. Интерфейс пользователя

Графический интерфейс приложения организован в виде трёх вкладок:

Вкладка 1: "Предметы и вопросы"

- Форма добавления нового предмета
- Таблица со списком всех предметов
- Форма добавления вопросов с 4 вариантами ответов
- Таблица вопросов выбранного предмета
- Кнопки удаления предметов и вопросов

Вкладка 2: "Учащиеся и результаты"

- Форма регистрации нового студента
- Таблица со списком всех учащихся
- Поиск студентов по фамилии
- Таблица результатов тестирования
- Отображение процентов успеваемости

Вкладка 3: "Проведение тестирования"

- Выбор студента из выпадающего списка
- Выбор предмета для тестирования
- Область отображения вопросов с вариантами ответов
- Интерактивный выбор ответов
- Автоматический подсчёт баллов и процентов

- Сохранение результатов в базу данных

4. Реализация системы

1. Класс KnowledgeControlSystem (Модуль knowledgecontrol.py)

Класс реализует всю бизнес-логику работы с данными системы.

Конструктор класса:

```
def __init__(self, db_path: str = "knowledge_control.db")
```

Инициализирует соединение с базой данных и создаёт таблицы.

Методы работы с предметами:

1. `create_tables()` - создаёт таблицы в базе данных
 - Создает таблицы: `students`, `subjects`, `results`
 - Не создает таблицы вопросов (они создаются при добавлении предмета)
2. `add_subject(subject_name: str) -> Tuple[bool, str]`
 - Параметры: `subject_name` - название предмета
 - Возвращает: кортеж (успех, сообщение)
 - Создает таблицу вопросов для предмета
 - Проверяет на дублирование
3. `get_all_subjects() -> List[Dict[str, Any]]`
 - Возвращает: список словарей с информацией о предметах
 - Сортировка по названию предмета
4. `get_subject_by_id(subject_id: int) -> Optional[Dict[str, Any]]`
 - Параметры: `subject_id` - ID предмета
 - Возвращает: словарь с информацией или `None`
5. `delete_subject(subject_id: int) -> Tuple[bool, str]`
 - Удаляет предмет, таблицу вопросов и все результаты

Методы работы с учащимися:

1. `add_student(first_name, last_name, group_name, birth_date, patronymic=None) -> Tuple[bool, str]`
 - Параметры:
 - `first_name` - имя учащегося
 - `last_name` - фамилия учащегося
 - `group_name` - название группы
 - `birth_date` - дата рождения (ГГГГ-ММ-ДД)
 - `patronymic` - отчество (необязательно)
 - Выполняет валидацию данных
 - Проверяет формат даты
2. `get_student(student_id: int) -> Optional[Dict[str, Any]]`

- Параметры: `student_id` - ID учащегося
- Возвращает: информацию о студенте или `None`
- 3. `get_all_students()` -> `List[Dict[str, Any]]`
 - Возвращает: список всех учащихся
 - Сортировка по фамилии и имени
- 4. `search_students_by_name(search_term: str)` -> `List[Dict[str, Any]]`
 - Параметры: `search_term` - поисковый запрос (фамилия, имя или отчество)
 - Возвращает: список найденных студентов
 - Поиск регистронезависимый (поддерживает поиск с маленькой и большой буквы)
 - Использует фильтрацию на стороне Python для корректной работы с кириллицей
 - Ищет совпадения в фамилии, имени и отчестве студента
- 5. `delete_student(student_id: int)` -> `Tuple[bool, str]`
 - Удаляет студента и все его результаты
- 6. `update_student(student_id, first_name, last_name, group_name, birth_date, patronymic=None)` -> `Tuple[bool, str]`
 - Параметры:
 - `student_id` - ID учащегося
 - `first_name` - имя учащегося
 - `last_name` - фамилия учащегося
 - `group_name` - название группы
 - `birth_date` - дата рождения (ГГГГ-ММ-ДД)
 - `patronymic` - отчество (необязательно)
 - Выполняет валидацию данных с проверкой реалистичности даты
 - Проверяет диапазон года (1900-2026)
 - Проверяет корректность дня для каждого месяца (с учётом високосных лет)
- 7. `validate_birth_date(birth_date: str)` -> `Tuple[bool, str]`
 - Параметры: `birth_date` - дата рождения в формате ГГГГ-ММ-ДД
 - Возвращает: кортеж (успех, сообщение)
 - Проверка формата даты
 - Проверка диапазона года: 1900-2026
 - Проверка диапазона месяца: 01-12
 - Проверка корректности дня для каждого месяца:
 - Январь, Март, Май, Июль, Август, Октябрь, Декабрь: 31 день
 - Апрель, Июнь, Сентябрь, Ноябрь: 30 дней
 - Февраль: 28 или 29 дней (високосный год)

Методы работы с вопросами:

1. `add_question(subject_id, question_text, variant_a, variant_b, variant_c, variant_d, correct_answer, question_number=None)` -> `Tuple[bool, str]`
 - Параметры:
 - `subject_id` - ID предмета
 - `question_text` - текст вопроса
 - `variant_a...variant_d` - варианты ответов

- correct_answer - правильный ответ ('A', 'B', 'C' или 'D')
 - question_number - номер вопроса (автоинкремент если не указан)
- 2. get_questions_by_subject(subject_id: int) -> List[Dict[str, Any]]
 - Возвращает: все вопросы для указанного предмета
- 3. get_question_by_id(subject_id: int, question_id: int) -> Optional[Dict[str, Any]]
 - Возвращает: конкретный вопрос по ID
- 4. delete_question(subject_id: int, question_id: int) -> Tuple[bool, str]
 - Удаляет вопрос из базы данных
- 5. update_question(subject_id, question_id, question_number, question_text, variant_a, variant_b, variant_c, variant_d, correct_answer) -> Tuple[bool, str]
 - Параметры:
 - subject_id - ID предмета
 - question_id - ID вопроса
 - question_number - номер вопроса
 - question_text - текст вопроса
 - variant_a...variant_d - варианты ответов
 - correct_answer - правильный ответ ('A', 'B', 'C' или 'D')
 - Обновляет данные вопроса в базе
 - Проверяет на дублирование номера вопроса

Методы работы с результатами:

1. save_result(student_id, subject_id, score, total_questions) -> Tuple[bool, str]
 - Параметры:
 - student_id - ID учащегося
 - subject_id - ID предмета
 - score - количество правильных ответов
 - total_questions - общее количество вопросов
 - Автоматически вычисляет процент правильных ответов
2. get_results_by_student(student_id: int) -> List[Dict[str, Any]]
 - Возвращает: все результаты тестирования студента
 - Сортировка по дате (новые сначала)
3. get_results_by_subject(subject_id: int) -> List[Dict[str, Any]]
 - Возвращает: все результаты по предмету
 - Включает информацию о студентах
4. has_student_taken_test(student_id: int, subject_id: int) -> bool
 - Проверяет, проходил ли студент тест по предмету
5. get_statistics_by_subject(subject_id: int) -> Dict[str, Any]
 - Возвращает статистику: количество тестов, средний/мин/макс процент

2. Класс KnowledgeControlApp (Модуль app_ui.py)

Класс реализует графический интерфейс приложения на базе Tkinter.

Конструктор класса:

```
def __init__(self, root: tk.Tk)
```

Инициализирует главное окно и создаёт виджеты.

Основные методы:

1. `create_widgets()` - создаёт основные виджеты приложения
 - Настраивает стили Tkinter
 - Создает контейнер вкладок (Notebook)
 - Инициализирует три вкладки
2. `setup_subjects_tab()` - настраивает вкладку предметов и вопросов
 - Создает формы добавления предмета и вопросов
 - Создает таблицы для отображения данных
 - Добавляет обработчики событий
3. `setup_students_tab()` - настраивает вкладку учащихся и результатов
 - Создает форму регистрации студента
 - Создает таблицу студентов
 - Добавляет поиск по фамилии
 - Создает таблицу результатов
4. `setup_testing_tab()` - настраивает вкладку проведения тестирования
 - Создает выпадающие списки для выбора студента и предмета
 - Создает область для отображения вопросов
 - Добавляет кнопки управления тестированием
5. `update_subjects_list()` - обновляет список предметов в таблице
6. `update_questions_list()` - обновляет список вопросов
7. `update_students_list()` - обновляет список студентов
8. `update_results_list(student_id)` - обновляет результаты для студента (с отображением ФИО)
9. `add_subject()` - обработчик добавления нового предмета
10. `add_student()` - обработчик добавления нового студента
11. `add_question()` - обработчик добавления нового вопроса
12. `edit_question()` - обработчик редактирования вопроса
 - Открывает диалоговое окно с полями вопроса
 - Позволяет изменить номер, текст, варианты ответов и правильный ответ
 - Корректно устанавливает модальность окна (использует `update_idletasks()`, `deiconify()`, `focus_force()` перед `grab_set()`)
 - Сохраняет изменения в базе данных
 - Обновляет таблицу вопросов после редактирования
13. `delete_question()` - обработчик удаления вопроса
14. `edit_student()` - обработчик редактирования данных студента
 - Открывает диалоговое окно с данными студента
 - Позволяет изменить ФИО, группу и дату рождения
 - Выполняет валидацию даты при сохранении (проверка диапазона 1900-2026, корректность дней месяца)
 - Корректно устанавливает модальность окна
 - Обновляет таблицу студентов и результатов после сохранения
15. `delete_student()` - обработчик удаления студента
 - Запрашивает подтверждение удаления
 - Удаляет студента и все его результаты
16. `start_test()` - начинает тестирование
17. `save_result()` - сохраняет результаты тестирования

18. `search_results_by_student()` - ищет результаты по фамилии (регистронезависимый поиск)
- Поддерживает поиск с маленькой и большой буквы (например, "иванов" и "Иванов")
 - При нахождении одного студента показывает его результаты
 - При нахождении нескольких выводит список для уточнения
19. `on_closing()` - обработчик закрытия приложения
- Корректно закрывает соединение с БД

3. Структура базы данных `knowledge_control.db`

Полный SQL код создания таблиц:

-- Таблица учащихся

```
CREATE TABLE IF NOT EXISTS students (  
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    patronymic TEXT,  
    group_name TEXT NOT NULL,  
    birth_date TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Таблица предметов

```
CREATE TABLE IF NOT EXISTS subjects (  
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    subject_name TEXT UNIQUE NOT NULL,  
    table_name TEXT UNIQUE NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Таблица результатов

```
CREATE TABLE IF NOT EXISTS results (  
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_id INTEGER NOT NULL,  
    subject_id INTEGER NOT NULL,  
    score INTEGER NOT NULL,  
    total_questions INTEGER NOT NULL,  
    percentage REAL NOT NULL,  
    test_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)  
);
```

-- Таблицы вопросов создаются динамически для каждого предмета

-- Пример для предмета "Математика":

```
CREATE TABLE IF NOT EXISTS questions_математика (  
    question_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    question_number INTEGER NOT NULL,  
    question_text TEXT NOT NULL,  
    variant_a TEXT NOT NULL,  
    variant_b TEXT NOT NULL,  
    variant_c TEXT NOT NULL,  
    variant_d TEXT NOT NULL,  
    correct_answer TEXT NOT NULL CHECK(correct_answer IN ('A', 'B', 'C', 'D')),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5. Запуск и работа программы

Запуск приложения

Windows:

python main.py

Linux/macOS:

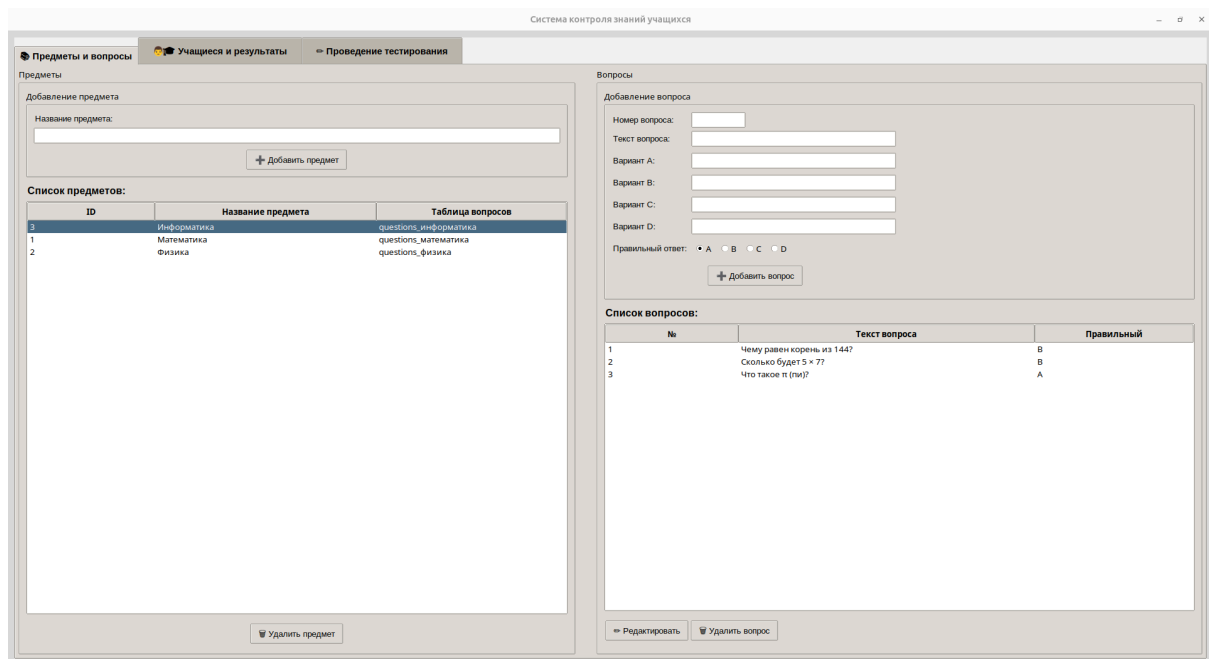
python3 main.py

При первом запуске автоматически создаётся файл базы данных knowledge_control.db.

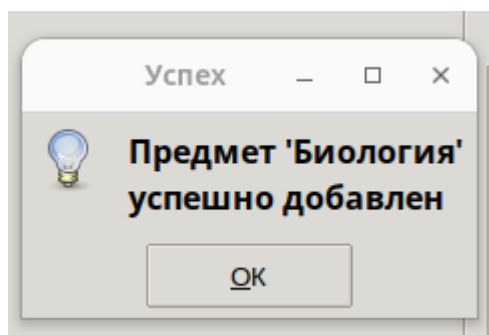
Вкладка 1 - Предметы и вопросы

Добавление нового предмета:

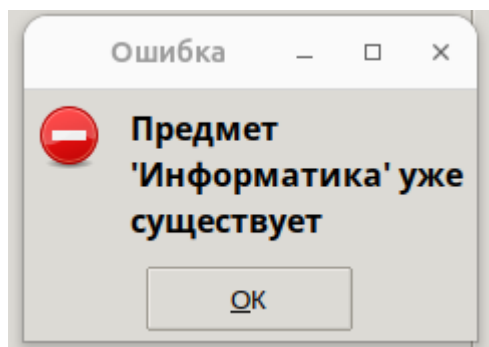
1. Введите название предмета в поле ввода (например, "Математика", "Физика", "Информатика")
2. Нажмите кнопку "Добавить предмет"
3. Система проверяет название на пустоту и дублирование
4. При успешном добавлении создаётся таблица вопросов для предмета
5. Появляется уведомление об успехе



[Рис.1 - Главный экран с вкладкой "Предметы и вопросы"]



[Рис.2 - Уведомление об успешном добавлении предмета]



[Рис.3 - Ошибка при дублировании предмета]

Добавление вопросов:

1. Выберите предмет из таблицы слева
2. Заполните форму добавления вопроса:
 - Номер вопроса (можно оставить пустым для автоинкремента)
 - Текст вопроса
 - 4 варианта ответа (A, B, C, D)

- Выберите правильный ответ переключателем
3. Нажмите "Добавить вопрос"
 4. Вопрос появляется в таблице справа

Вопросы

Добавление вопроса

Номер вопроса:

Текст вопроса:

Вариант А:

Вариант В:

Вариант С:

Вариант D:

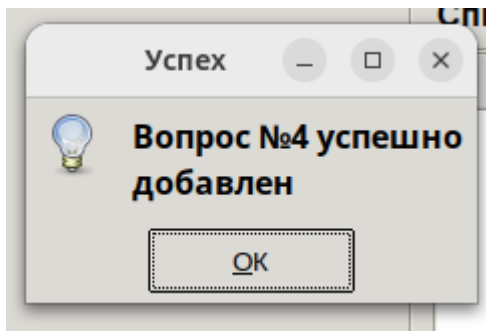
Правильный ответ: ☒ А ☐ В ☐ С ☐ D

[Рис.4 - Форма добавления вопроса]

Список вопросов:

№	Текст вопроса	Правильный
1	Чему равен корень из 144?	В
2	Сколько будет 5×7 ?	В
3	Что такое π (пи)?	А

[Рис.5 - Таблица с вопросами по предмету]



[Рис.6 - Уведомление об успешном добавлении вопроса]

Примеры вопросов для предмета "Математика":

- Вопрос 1: "Чему равен корень из 144?"
 - A) 10
 - B) 12 ✓
 - C) 14
 - D) 16
- Вопрос 2: "Сколько будет 5×7 ?"
 - A) 30
 - B) 35 ✓
 - C) 42
 - D) 28

Вкладка 2 - Учащиеся и результаты

Добавление студентов:

1. Заполните форму:
 - Фамилия (обязательно)
 - Имя (обязательно)
 - Отчество (необязательно)
 - Группа (обязательно)
 - Дата рождения в формате ГГГГ-ММ-ДД (обязательно)
2. Нажмите "Добавить студента"
3. Студент появляется в таблице

Система контроля знаний учащихся

Предметы и вопросы **Учащиеся и результаты** Проведение тестирования

Добавление студента

Фамилия:

Имя:

Отчество:

Группа:

Дата рождения (ГГГГ-ММ-ДД):

Список студентов

Поиск по фамилии:

ID	Фамилия	Имя	Отчество	Группа	Дата рождения
1	Иванов	Иван	Иванович	ПИ-201	2000-05-15
2	Петрова	Мария	Сергеевна	ПИ-201	2001-03-22

Результаты тестирования

ФИО студента	Предмет	Баллы	Всего вопросов	Процент	Дата теста
--------------	---------	-------	----------------	---------	------------

[Рис.7 - Вкладка "Учащиеся и результаты"]

Добавление студента

Фамилия:

Имя:

Отчество:

Группа:

Дата рождения (ГГГГ-ММ-ДД):

[Рис.8 - Форма добавления студента]

Список студентов

Поиск по фамилии:

ID	Фамилия	Имя	Отчество	Группа	Дата рождения
1	Иванов	Иван	Иванович	ПИ-201	2000-05-15
2	Петрова	Мария	Сергеевна	ПИ-201	2001-03-22

[Рис.9 - Таблица со списком студентов]

Примеры данных студентов:

- Иванов Иван Иванович, группа ПИ-201, дата рождения: 2000-05-15
- Петрова Мария Сергеевна, группа ПИ-201, дата рождения: 2001-03-22

Поиск результатов по студенту:

1. Введите фамилию в поле поиска
2. Нажмите "Найти"
3. При нахождении одного студента - показываются его результаты с ФИО в таблице
4. При нахождении нескольких - выводится список для уточнения

Система контроля знаний учащихся

Чт, 9 апреля 15:01

Предметы и вопросы | Учащиеся и результаты | **Проведение тестирования**

Добавление студента

Фамилия:

Имя:

Отчество:

Группа:

Дата рождения (ГГГГ-ММ-ДД):

Список студентов

Поиск по фамилии:

ID	Фамилия	Имя	Отчество	Группа	Дата рождения
1	Иванов	Иван	Иванович	ПИ-201	2000-05-15
2	Петрова	Мария	Сергеевна	ПИ-201	2001-03-22

Резу...

Найден студент:
Иванов Иван

Результаты тестирования

ФИО студента	Предмет	Баллы	Всего вопросов	Процент	Дата теста
Иванов Иван Иванович	Информатика	2	3	66.67%	2026-04-09 11:30:43

[Рис.10 - Поиск результатов по студенту]

ФИО студента	Предмет	Баллы	Всего вопросов	Процент	Дата теста
Иванов Иван Иванович	Информатика	2	3	66.67%	2026-04-09 11:30:43

[Рис.11 - Таблица с результатами тестирования]

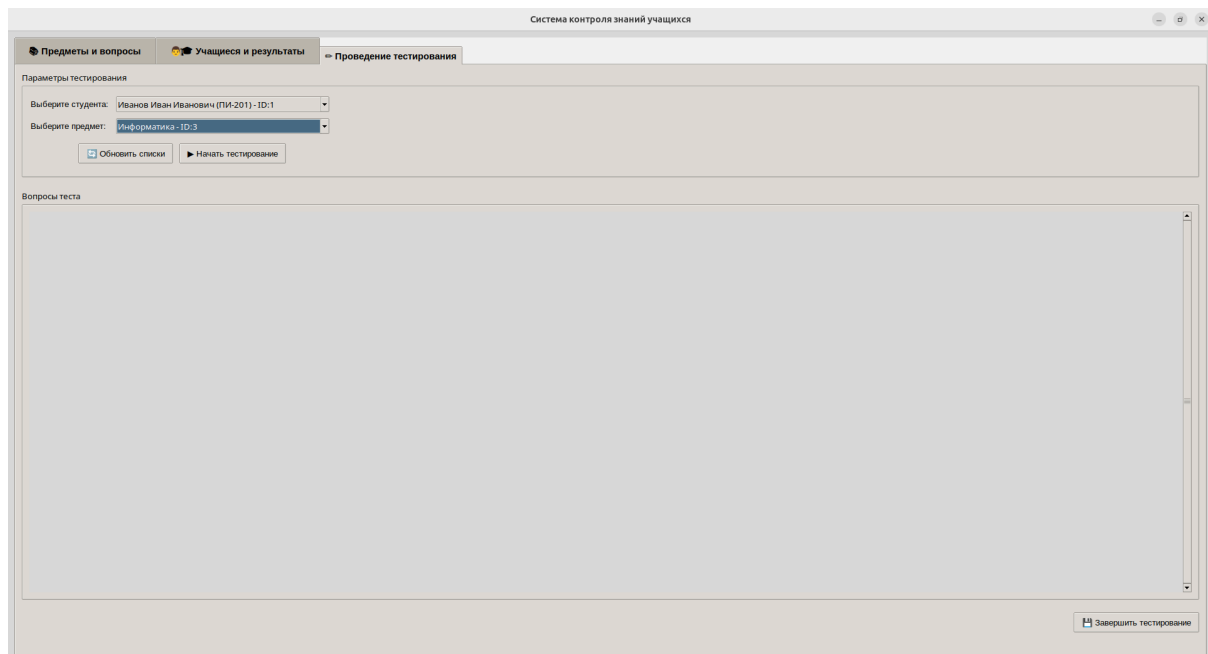
Примеры результатов:

Предмет	Баллы	Всего	Процент	Дата
Математика	9	10	90.0%	2024-01-15
Физика	7	10	75.0%	2024-01-16
Информатика	10	10	100.0%	2024-01-17

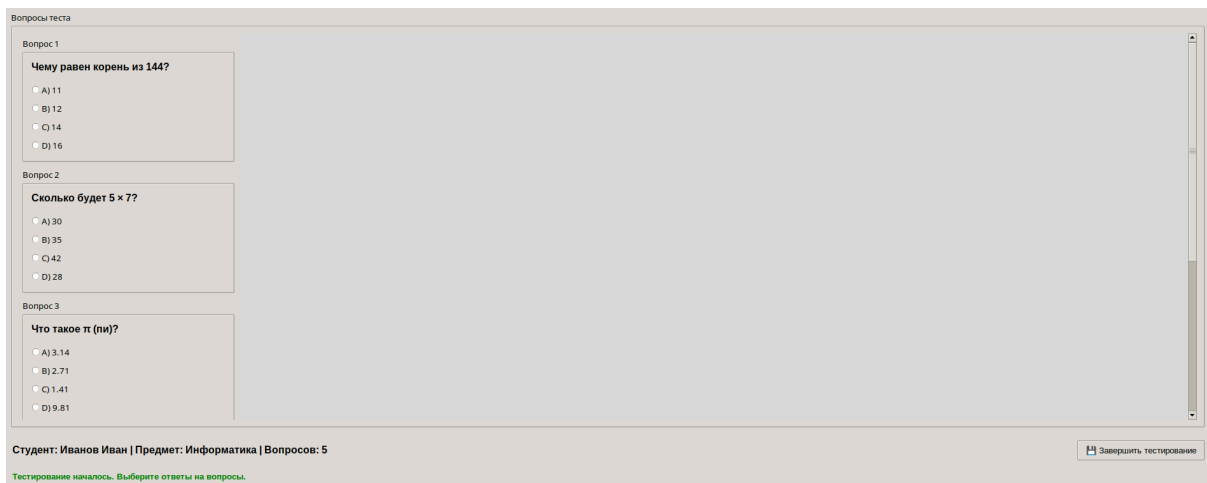
Вкладка 3 - Проведение тестирования

Процесс тестирования:

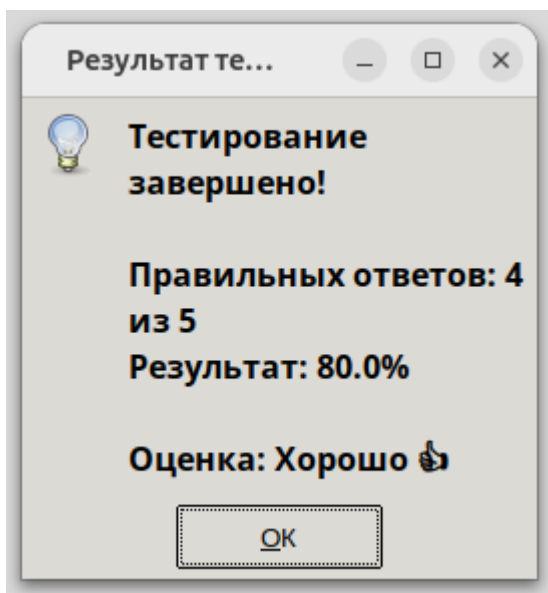
1. Нажмите "Обновить списки" для загрузки актуальных данных
2. Выберите студента из выпадающего списка
3. Выберите предмет для тестирования
4. Нажмите "Начать тестирование"
5. Система загружает вопросы и отображает их с вариантами ответов
6. Студент выбирает ответы на каждый вопрос
7. Нажмите "Завершить тестирование"
8. Система подсчитывает баллы и процент правильных ответов
9. Результат сохраняется в базу данных



[Рис.12 - Вкладка "Проведение тестирования"]



[Рис.13 - Процесс прохождения тест]



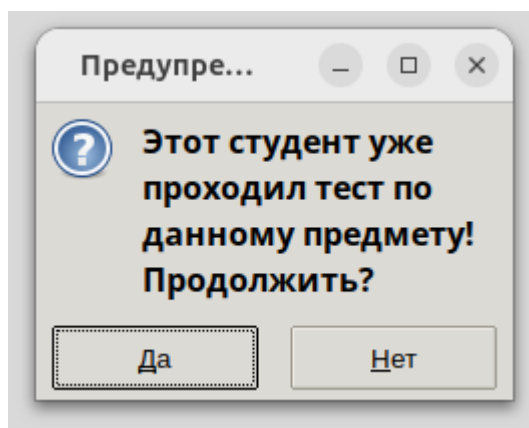
[Рис.14 - Результаты теста с процентами]

Система оценивания:

- 90-100%: Оценка "Отлично"
- 75-89%: Оценка "Хорошо"
- 50-74%: Оценка "Удовлетворительно"
- Менее 50%: Оценка "Неудовлетворительно"

Защита от повторного прохождения:

- При попытке повторно пройти тест система выдаёт предупреждение
- Пользователь может подтвердить или отменить действие



[Рис.15 - Ошибка при попытке повторного прохождения]

Удаление вопросов:

1. Выберите вопрос в таблице
2. Нажмите "Удалить вопрос"
3. Подтвердите удаление в диалоговом окне

6. Заключение

В ходе выполнения работы была разработана полнофункциональная система автоматизированного контроля знаний учащихся на языке программирования Python с использованием базы данных SQLite и графической библиотеки Tkinter.

Основные достижения:

1. Создана архитектура приложения с чётким разделением бизнес-логики и пользовательского интерфейса
2. Реализована динамическая структура базы данных с автоматическим созданием таблиц вопросов для каждого предмета
3. Разработан интуитивно понятный графический интерфейс с тремя основными вкладками
4. Обеспечена полная валидация входных данных и обработка ошибок
5. Реализована система автоматического подсчёта результатов с процентным отображением
6. Предусмотрена защита от повторного прохождения тестов

Практическая значимость: Разработанная система может быть использована в образовательных учреждениях для:

- Проведения промежуточной и итоговой аттестации
- Организации самостоятельной работы студентов
- Мониторинга успеваемости учащихся
- Снижения нагрузки на преподавателей при проверке тестов

Технические характеристики готового продукта:

- Язык программирования: Python 3.7+
- База данных: SQLite 3

- GUI: Tkinter с темой 'clam'
- Объём кода: ~1400 строк
- Количество классов: 2 основных
- Количество методов: 40+
- Формат хранения данных: локальный файл .db

Перспективы развития:

- Добавление экспорта результатов в форматы Excel/PDF
- Реализация сетевого режима работы
- Создание системы генерации случайных вариантов тестов
- Добавление мультимедийных вопросов (с изображениями, аудио, видео)
- Интеграция с системами дистанционного обучения